Python for Data Science ¶

Python

- Multi-platform
- Interpreted / Dynamic Typing
- Garbage Collected
- Object-Oriented & Procedural
- **Large Standard Library**

Created by Guido van Rossum (https://www.linkedin.com/in/guido-van-rossum-4a0756/), first released in **1991**

REPL

Read, Eval, Print, Loop.

Open your terminal and launch:

```
ipython
```

Documentation

👉 The Python 3.12 Standard Library (https://docs.python.org/3.12/library/index.html)

Built-in Types

Let's look at the important sections of the documentation (https://docs.python.org/3.12/library/stdtypes.html) and explore the **simple** built-in types.

Numeric Types 🔗 (https://docs.python.org/3.12/library/stdtypes.html#numeric-types-int-float-complex)

```
In [ ]: type(42)
```

```
Out[ ]: int
```

```
In [ ]: type(3.14)
```

```
Out[ ]: float
```

In [ ]:
```
4 + 9.5
```
Out[ ]: 13.5

In [ ]:
```
abs(-5)
```
Out[ ]: 5

Booleans 🔗 (https://docs.python.org/3.12/library/stdtypes.html#truth-value-testing)

In [ ]:
```
type(True) # or type(False)
```
Out[ ]: bool

In [ ]:
```
True and False
```
Out[ ]: False

In [ ]:
```
True or False
```
Out[ ]: True

In [ ]:
```
not True
```
Out[ ]: False

What are falsy values in Python?

- Constants defined to be false: `None` and `False`
- Zero of any numeric type: `0` , `0.0` , etc.
- Empty sequences and collections: `''` , `()` , `[]` , `{}` , `set()` , `range(0)`

Comparisons

In [ ]:
```
5 < 3 # Strictly less than
```
Out[ ]: False

In [ ]:
```
4 <= 4 # Less than or equal
```
Out[ ]: True

In [ ]:
```
"boris" == "seb" # equal, note the `double` =
```
Out[ ]: False

In [ ]:
```
"boris" != "seb" # not equal
```
Out[ ]: True

Text Sequence Type (Strings, `str`)

In [ ]:
```
"boris"
```

Out[ ]:
```
'boris'
```

In [ ]:
```
"boris".capitalize()
```

Out[ ]:
```
'Boris'
```

In [ ]:
```
"   boris ".strip()
```

Out[ ]:
```
'boris'
```

In [ ]:
```
type(int("42"))
```

Out[ ]:
```
int
```

Check out the documentation (https://docs.python.org/3.12/library/stdtypes.html#string-methods) for more `str` methods.

Advanced built-in types

Sequence Types

- `list` (https://docs.python.org/3.12/library/stdtypes.html#list)
- `tuple` (https://docs.python.org/3.12/library/stdtypes.html#tuple)
- `range` (https://docs.python.org/3.12/library/stdtypes.html#range)

An example of `list`

In [ ]:
```
["apple", "cherries", "banana"]
```

Out[ ]:
```
['apple', 'cherries', 'banana']
```

In [ ]:
```
len(["apple", "cherries", "banana"])
```

Out[ ]:
```
3
```

In [ ]:
```
"banana" in ["apple", "cherries", "banana"]
```

Out[ ]:
```
True
```

Mapping Type

There is just one in Python, `dict` (read "dictionary").

In [ ]:
```
{'city': 'Paris', 'population': 2_141_000}
```

Out[ ]:
```
{'city': 'Paris', 'population': 2141000}
```

In [ ]:
```
len({'city': 'Paris', 'population': 2_141_000})
```

Out[ ]:
```
2
```

In [ ]:
```
'city' in {'city': 'Paris', 'population': 2_141_000}
```

Out[ ]:
```
True
```

Variables

In [ ]:
```
name = 'John' # Assignment
print(name)
```

```
John
```

In [ ]:
```
name = 'Paul' # Re-assignment
print(name)
```

```
Paul
```

In [ ]:
```
first_name = 'John'
last_name = 'Lennon'
```

In [ ]:
```
full_name = first_name + last_name
print(full_name)
```

```
JohnLennon
```

String Formatting - Interpolation

In [ ]:
```
sentence = f'Hi, my name is {first_name} {last_name}'
print(sentence)
```

```
Hi, my name is John Lennon
```

`list` operations

In [ ]:
```
# Indexes:    0       1       2
beatles = ['john', 'paul', 'ringo']
```

In [ ]:
```
beatles[0] # Read
```

Out[ ]:
```
'john'
```

In [ ]:
```
beatles.append('george') # Create (in place)
print(beatles)
```

```
['john', 'paul', 'ringo', 'george']
```

In [ ]:
```
beatles[1] = 'Paul' # Update (in place)
print(beatles)
```

```
['john', 'Paul', 'ringo', 'george']
```

Other `list` operations

In [ ]:
```
beatles
```

Out[ ]:
```
['john', 'Paul', 'ringo', 'george']
```

In [ ]:
```
beatles[1:3] # Slice of 1 to 3 (excluded)
```

Out[ ]:
```
['Paul', 'ringo']
```

In [ ]:
```
del beatles[1] # Delete (in place)
print(beatles)
```

```
['john', 'ringo', 'george']
```

`dict` operations

In [ ]:
```
instruments = {'john': 'guitar', 'paul': 'bass'} # Keys are `str`, Values too
```

In [ ]:
```
instruments['john'] # Read
```

Out[ ]:
```
'guitar'
```

In [ ]:
```
instruments['ringo'] = 'drums' # Create / Update
print(instruments)
```

```
{'john': 'guitar', 'paul': 'bass', 'ringo': 'drums'}
```

In [ ]:
```
del instruments['john']
instruments
```

Out[ ]:
```
{'paul': 'bass', 'ringo': 'drums'}
```

Other `dict` operation

In [ ]:
```
instruments
```

Out[ ]:
```
{'paul': 'bass', 'ringo': 'drums'}
```

What will happen if we run `instruments['john']` ?

In [ ]:
```
instruments['john']
```

```
---------------------------------------------------------------------
--------
KeyError                                  Traceback (most recent ca
ll last)
<ipython-input-40-4f6ae9ee253b> in <module>
----> 1 instruments['john']

KeyError: 'john'
```

In [ ]:
```
instruments.get('john', 'Default instrument')
```

Out[ ]:
```
'Default instrument'
```

Control Flow

Once again, the documentation is very thorough (https://docs.python.org/3.12/tutorial/controlflow.html).

`if` statement

In [ ]:
```
age = int(input("How old are you?"))

if age >= 21:
    print("You can become president")
elif age >= 18:
    print("You can vote")
else:
    print("Be patient!")
```

`for` statement (on a `list`)

In [ ]:
```
words = ['cat', 'wolf', 'beetle']
for word in words:
    print(word.upper())
```

```
CAT
WOLF
BEETLE
```

List comprehension

You can achieve the same result in just one line:

In [ ]:
```
[print(word.upper()) for word in words]
```

... or store it in a new variable:

In [ ]:
```
uppercase_words = [word.upper() for word in words]
print(uppercase_words)
```

```
['CAT', 'WOLF', 'BEETLE']
```

What if you need the index inside the `for` loop?

In [ ]:
```
for index, word in enumerate(words):
    print(index, word)
```

```
0 cat
1 wolf
2 beetle
```

`for` statement (on a `dict`)

In [ ]:
```
instruments
```

Out[ ]:
```
{'paul': 'bass', 'ringo': 'drums'}
```

In [ ]:
```
for beatle, instrument in instruments.items():
    print(f'{beatle.capitalize()} plays the {instrument}')
```

```
Paul plays the bass
Ringo plays the drums
```

`while` loop

In [ ]:
```
i = 1
while i <= 4:
    print(i)
    i = i + 1
```

```
1
2
3
4
```

Functions

Let's see how we can define and use our own functions (https://docs.python.org/3.12/tutorial/controlflow.html#defining-functions).

In [ ]:
```
def is_even(number):
    return number % 2 == 0
```

- The function is named `is_even`
- The function takes **1** parameter: `number`
- The function returns a `bool`

In [ ]:
```
result = is_even(42)
print(result)
```

```
True
```

- We called the function with the **argument** `42` .
- The returned value is stored in a `result` variable.

Keyword Arguments

Let's have a look at `pandas.DataFrame.from_dict` (https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.from_dict.html) (Source (https://github.com/pandas-dev/pandas/blob/v1.0.3/pandas/core/frame.py#L1168-L1247)). Some arguments are passed with the syntax `parameter=value` .

In [ ]:
```
def is_odd(number=0):
    return number % 2 == 1
```

In [ ]:
```
result = is_odd(number=1)
print(result)
```

```
True
```

Mixing positional and keyword arguments

In [ ]:
```
def full_name(first_name, last_name, capitalize=False):
    if capitalize:
        return f"{first_name.capitalize()} {last_name.capitalize
()}"
    else:
        return f"{first_name} {last_name}"
```

In [ ]:
```
print(full_name("john", "lennon"))
print(full_name("ringo", "starr", capitalize=True))
```

```
john lennon
Ringo Starr
```

Modules 🔗 (https://docs.python.org/3.12/tutorial/modules.html)

As our programs grow longer, we'll want to split the code into multiple files and folders.

Here is a very simple example to illustrate this technique:

```
mkdir project && cd project
touch app.py
mkdir helpers && touch helpers/__init__.py
```

In [ ]:
```
# helpers/__init__.py
import random
import string

def generate_password(size):
    """Generate a random lowercase ASCII password of given size"""
    letters = string.ascii_lowercase
    return ''.join(random.choice(letters) for i in range(size))
```

In [ ]:
```
# app.py
from helpers import generate_password # Module import here

print(generate_password(16))
```

```
python app.py
```

Before you go, one more thing!

Debugging 🐛

- `print(your_variable)`
- `ipython`
- `python <file_name>`

When running your code with `python <file_name>` you can use the IPython debugger (ipdb) (https://ipython.readthedocs.io/en/stable/) thanks to the built-in `breakpoint()` (https://docs.python.org/3.12/library/functions.html#breakpoint) function.

It will let you investigate the current state of your code.

Let's try with our `generate_password` function!

Add the debugger in the function definition and run `python app.py` file from the Terminal.

In [ ]:
```
# helpers/__init__.py
import random
import string

def generate_password(size, upper=False):
    """Generate a random lowercase ASCII password of given size"""
    letters = string.ascii_uppercase if upper else string.ascii_low
ercase
    breakpoint() # <- That's a BREAKPOINT!
    return ''.join(random.choice(letters) for i in range(size))
```

You can access the `args`, variables defined within the function (`letters`) and much more: just type the name of the variable and hit ENTER.

To see the part of the code you're at, type `list` (or just `l`).

To continue the execution try:

- `next` (or just `n`) - execute the current line and go to the next line
- `step` (or just `s`) - step into the next function that will be called
- `continue` (or just `c`) - continue until the next `breakpoint()`, or the end

If you want to see all available commands type `help` (or just `h`).

Tips from TAs ❤️

Read the instructions!

It's *OK* not to finish all challenges

Pseudocode first!

Programming is all about going step by step, decomposing hard problems into small ones.

🔪 Write down the steps you'll need to code in plain English.

Separate 🤔 thinking about the ⚙️ logic from getting the `["syntax')}` exactly right.

Run your code!

‼️ Don't try to `make` directly.

```
# example.py
def my_function(param1, param2):
    # BODY
    # BODY
    # BODY
    return # SOMETHING

print(my_function(arg1, arg2)) # <- Temp code to test! Remove before fina
l commit/push
```

and then run the code in your terminal:

```
python example.py
```

Debug your code!

Poor man's way: `print()` all the things! What's the type of this variable? What does it contain at this point of the program?

Better: debug your code using `breakpoint()`!

💪 **Debugging** is one of the most important skills to master in your career!

Advanced Language Topics

To master Python, here are some additional topics to dive into during the Bootcamp:

- List Comprehensions (https://treyhunner.com/2015/12/python-list-comprehensions-now-in-color/)
- Iterators & Generators (https://anandology.com/python-practice-book/iterators.html)
- Object Oriented Programming (https://anandology.com/python-practice-book/object_oriented_programming.html)
- `*args` & `**kwargs` (https://www.digitalocean.com/community/tutorials/how-to-use-args-and-kwargs-in-python-3)
- Memory Management / Garbage Collection (https://docs.python.org/3.12/faq/design.html?highlight=garbage%20collector#how-does-python-manage-memory)

Your turn!

Head to Kitt, find your **buddy** and start your first challenge!

Remember:

- Read the assignment **carefully**
- Run your code with `python <file.py>` before running `make`
- Talk with your buddy
- Raise a **ticket** 🙋

Have fun! 🚀